NEXUS-R Roadmap

> Verified milestones. No speculative dates.

Phase 1 — Core Runtime (COMPLETE)

Goal: A user can give a natural language task and have it executed locally or via BYOK, with full audit trail.

Delivered:

- CLI tool (`nexus run`, `history`, `cost`, `config`)
- 2-tier CAR (local via Ollama, BYOK via LiteLLM)
- Terminal + filesystem sandbox (Docker/subprocess)
- Event-sourced SQLite database (append-only, causal chaining)
- Cost tracking per request
- T1–T2 permission enforcement (deny-first)

Validation:

- Real Ollama integration (`qwen2.5:1.5b-instruct`)
- 20-task batch: 100% success, ~1.9s avg latency
- 100 concurrent tasks: zero failures
- EventStore: 0.024ms/append batched
- Test coverage: >80%

Gate: Passed. See `runtime_truthfulness_report_phase1_5.md`.

Phase 1.5 — Runtime Hardening (COMPLETE)

Goal: Prove the runtime is operationally trustworthy before building features.

Delivered:

- Real provider execution layer (not mocked)
- Streaming responses with cancellation
- Structured telemetry (JSON spans, timing metrics)
- Concurrency stress testing (20/50/100 tasks)
- Failure injection (provider crash, network loss, SQLite lock)
- Session recovery architecture (crash-resilient)
- Sandbox file traversal hardening

Validation:

- Real inference: validated

- Streaming: validated
- Concurrency: validated
- Recovery: validated (with residual cross-store atomicity gap)

Gate: Passed with noted limitations. See `known_limitations_phase1.md`.

Phase 2 — ETD + Full CAR (NEXT)

Goal: Does ETD measurably reduce cost and latency on repeated tasks? Do users feel in control?

Planned Deliverables:

ETD Pipeline (Core Innovation)

- **Trace recording** — Immutable execution logs from Phase 1 foundation
- **Distillation** — Extract minimal causal chain, drop dead-ends
- **Generalization verification** — Test on held-out variants, admit only >90% success
- **Parameterization** — Replace concrete values with typed slots
- **Deduplication** — Merge workflows with >80% step overlap
- **Retrieval** — Cosine similarity on intent embeddings + context gating
- **Conservative application** — Opt-in, environment match, abandon on failure
- **Invalidation** — Auto-degrade if failure rate >30% or 180 days unused

Full 4-Tier CAR

- 6-tier model chain: local_7b → local_14b → local_70b → BYOK_budget → BYOK_frontier → managed_premium
- Parallel probe — Send to T1+T2 simultaneously, discard expensive if cheap succeeds
- De-escalation learning — Update classifier when T1 handles T2-classified tasks
- Adaptive capability profiles — Learn from observed outcomes, not static specs

Trust Layer Expansion

- T3 permissions — User prompt for arbitrary terminal, external files, HTTP POST/PUT/DELETE, browser actions, API keys
- T4 permissions — Explicit confirmation + documented reason for delete files, access secrets, deploy to prod, financial transactions
- ML risk classifier — Rule-based initially, refined with observed outcomes
- Prompt injection defense — Input framing, output validation, action allowlisting, anomaly detection

Cost Dashboard (Web UI)

- FastAPI backend with REST endpoints
- HTML/JS frontend: cost trends, audit log, ETD stats
- WebSocket real-time updates during task execution
- Read-only audit view

Validation Gate:

- A/B test: 10 users with ETD enabled vs. 10 without

- Target: >40% cost reduction on repeated tasks
- Target: >60% latency reduction on cached workflows
- If ETD does not demonstrate >40% savings: **kill the project**

Phase 3 — Browser + Polish

Goal: Full end-to-end capability: terminal + files + browser + APIs. ETD composition. Three-tier memory fully operational.

Planned Deliverables:

Browser Automation

- Playwright via MCP, isolated Chromium instance
- Navigate, click, type, screenshot, extract
- No host filesystem access, no real browser profile
- T3 approval required for browser actions
- Session management for authenticated websites (see Phase 3.5 below)

ETD Composition

- Chain multiple ETDs: output of ETD1 feeds into input of ETD2
- Schema validation between chained workflows
- Track composition as new ETD with provenance

Three-Tier Memory

- **Volatile** — Working State (in-memory, dies with task)
- **Durable** — Event Store (SQLite, append-only, 90 days)
- **Identity** — User preferences, tool configs, API key refs, behavioral policies, classifier weights (encrypted, slow-changing)
- Vector embeddings for semantic search across execution history
- Deep Memory compaction (~71% token reduction target)

Execution Replay + Fork

- Step-by-step reconstruction from EventStore causal chain
- Fork from any step in past execution
- Interactive timeline visualization

Polished Web UI

- Dark mode, mobile-responsive
- Task submission from web UI
- ETD composition builder (drag-and-drop workflow chaining)
- User preferences editor
- Execution replay visualization

Validation Gate:

- Browser automation succeeds on >80% of test scenarios
- ETD composition reduces cost by >60% on multi-step workflows
- Personalization measurably improves routing accuracy over time

---

Phase 3.5 — Authenticated Session Management (Extension)

Goal: Persistent login sessions for websites requiring authentication.

Planned Deliverables:

- One-time T4 setup: `nexus auth add gmail` — user manually logs in, agent captures cookies
- Encrypted session storage (`~/.nexus-r/sessions/{service}.enc`)
- T2 auto-reuse for subsequent tasks with `--session` flag
- Session health check before task execution
- 30-day auto-expiry, manual invalidation
- Privacy rules: session data never exposed to LLM context, never logged, never included in ETD

Not in original PDF spec. Added based on user requirement for authenticated web workflows.

Phase 4 — Community + Scale

Goal: Community ETD exchange, multi-provider premium tier, advanced CAR, third-party API, mobile companion.

Planned Deliverables:

ETD Community Exchange

- Opt-in sharing with privacy stripping (remove paths, secrets, personal patterns)
- Verification pipeline for community ETDs (>80% generalization threshold)
- Rate limiting: 100 uploads/day, 1000 downloads/day
- Revenue share model for popular workflow authors

Advanced CAR

- Fine-tuned task classifier (logistic regression on execution history)
- Per-model accuracy tracking by task type
- Explainable routing decisions (which features contributed)

Third-Party API

- REST API for external integrations (`POST /api/v1/tasks`)
- API key authentication, rate limiting (1000 req/hour)
- OpenAPI/Swagger documentation

Mobile Companion

- View audit log, approve escalations, cost alerts
- Push notifications (Firebase Cloud Messaging)
- Quick approval for T3/T4 tasks from phone

Validation Gate:

- ETD accumulation shows >60% cost reduction after 50+ tasks
- Community ETDs are verified and applicable
- Mobile approval workflow completes in <10 seconds
- No security regressions from Phase 1–3

Anti-Roadmap (Explicitly Out of Scope)

These were considered and rejected in the v8.0 specification:

Feature	Why Rejected
Model Council (debating among models)	Too expensive, no proven value
World Model (complete state representation)	Open research problem, not engineering
Formal Verification Layer	Does not scale; deny-first + ML risk preferred
Custom Provider Abstraction	LiteLLM already handles 100+ providers
Separate Search/Retrieval Layer	Search is a tool via MCP, not a subsystem
12-Layer Architecture	Complexity is the enemy of shipping
AGI / Sovereign Cognition	Users buy outcomes, not concepts

Kill Criteria

The project should be halted if any of the following are not met at their respective gates:

Gate	Criterion	Action
Phase 1	>80% success rate on 20 tasks	Redesign execution sandbox
Phase 1	Average cost <$0.10 for T1–T2	Revisit routing logic
Phase 2	ETD shows >40% cost reduction on repeated tasks	**Kill project**
Phase 2	>90% generalization success rate for admitted ETDs	Raise threshold or narrow scope
Phase 3	Browser automation >80% success	Re-evaluate MCP tool integration
Any phase	Zero Critical/High security findings	Fix before proceeding

---

> A focused 6-module CLI that demonstrably saves users money is worth more than a 12-layer diagram that never compiles.